7 Using dependency injection to manage services

This chapter covers

- Understanding the role of dependency injection
- Examining dependency injection in ASP.NET Core
- Creating and using your own services
- Managing service lifetime

Dependency injection (DI) is a software engineering technique included as a feature in many web frameworks these days. Its *raison d'être* is enabling *loose coupling* between software components, resulting in code that is less brittle, more adaptable to change, easier to maintain, and easier to test. If you have worked with dependency injection before, all of this will feel familiar to you. In case dependency injection is a new concept to you, this chapter will help by explaining what it is and why you should care.

DI is at the heart of ASP.NET Core. The entire framework uses the built-in DI feature to manage its own dependencies, or *services*. Services are globally registered within a *container* when the application starts up and then provided by the container when needed by consumers. You encountered the main entry point to the service container when you looked at Program.cs in chapter 2. It is accessed via the `Services` property of the `WebApplicationBuilder`. You will recall that services that comprise the Razor Pages framework are registered with the container via the `AddRazorPages` method: